Transfer Learning with Pretrained CNNs

How much of the network do you actually need to retrain?

1. Introduction

To train a Convolutional Neural Network (CNN) from the ground up, you need two things that most people don't have: millions of labeled images and a lot of computing power. Transfer learning lets you get around both problems by using a network that has already been trained on a large dataset, usually ImageNet, which has 1.2 million images, each in one of 1,000 categories.

This tutorial will teach you what transfer learning is and how it works. You will know which layers of a network should be frozen and which ones should be fine-tuned by the end of this tutorial. This will depend on how many images you have compared to the original dataset. We will use the Oxford 102 Flowers dataset and ResNet-18 as an example.

2. Why Transfer Learning Works

There is a hierarchy of features learned by CNNs (convolutional neural networks). Yosinski et al. (2014) provide empirical evidence of this hierarchy whereby the early layers learn to recognize general, low-level features of images such as edges, gradients of color, and simple textures, while the later layers encode patterns that are more specific to a particular task. This finding is critical for enabling transfer learning to occur.

Layer depth	What it learns	Transferability
Layers 1–2	Edges, colour blobs, simple textures	High (universal across all image tasks)
Layers 3–4	Shapes, object parts, patterns	Medium (depends on domain similarity)
Final layers	Task-specific combinations	Low (tied to original classes)

Table 1: Feature hierarchy in a pretrained CNN and the transferability of each stage.

Since the early layers are already effective at detecting edges and textures, which are important to most any visual task, there is no need to retrain them on a small dataset. Indeed, doing so would be counterproductive because it would introduce noise. The question then becomes how deep you go into the network to unfreeze the weights.

3. Three Fine-Tuning Strategies

There is no single correct answer, but three strategies cover the most common scenarios:

Strategy A: Feature Extraction (freeze everything)

All of the convolutional layers are locked. The only part that is changed and trained is the last fully connected classification head. The CNN works as a fixed feature extractor. This works well when your data set is small and similar to ImageNet.

Strategy B: Partial Fine-Tuning (freeze early, unfreeze late)

The early convolutional layers are frozen, but the last one or two residual blocks and the

classification head are retrained. This is the most common approach in practice; it balances the stability of pretrained low-level features with the flexibility to adapt high-level representations to your domain.

Strategy C: Full Fine-Tuning (unfreeze everything)

During training, all weights are changed, usually with a very low learning rate so that the pretrained features don't get ruined. When you have a lot of data or your images are very different from ImageNet (like medical scans or satellite images), this is the right thing to do.

	Small dataset (< ~1k images)	Large dataset (> ~10k images)
Similar to ImageNet (natural photos)	Strategy A: Feature extraction	Strategy B or C: Full fine-tune
Different from ImageNet (medical, satellite)	Strategy B: Partial fine-tune	Strategy C: Full fine-tune

Table 2: Strategy selection guide based on dataset size and domain similarity.

4. Demonstration: ResNet-18 on Oxford 102 Flowers

Dataset

There are 8,189 photos in 102 flower categories in the Oxford 102 Flowers dataset (Nilsback & Zisserman, 2008). Many categories appear to be visually similar, making it a difficult, fine-grained classification problem. Importantly, it is an honest test of transfer learning across a domain gap because flowers are not included in ImageNet's 1,000 classes.

Model: ResNet-18

ResNet-18 (He et al., 2016) is a residual network with 18 layers. The first four layers are convolutional blocks, and the last layer is a global average pooling layer with a fully connected head. Because of its skip connections, gradients can flow smoothly during fine-tuning. This makes it a great base model for transfer learning tests.

Experiment setup

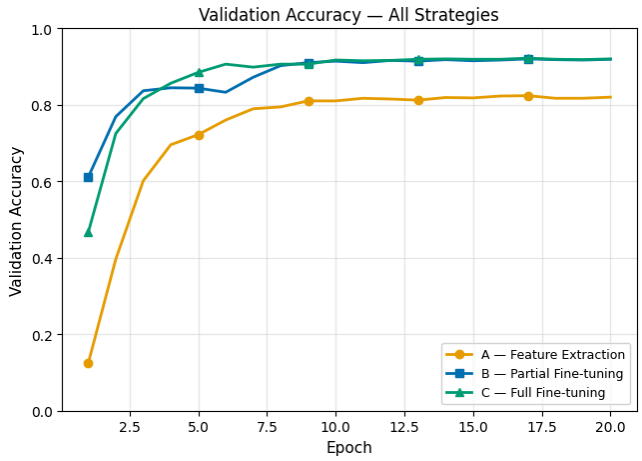
All three strategies use the same training setup to make sure the comparison is fair: SGD optimizer with momentum 0.9, cosine learning rate decay, 20 training epochs, and standard ImageNet normalization. The only thing that changes is which layers have requires_grad = True.

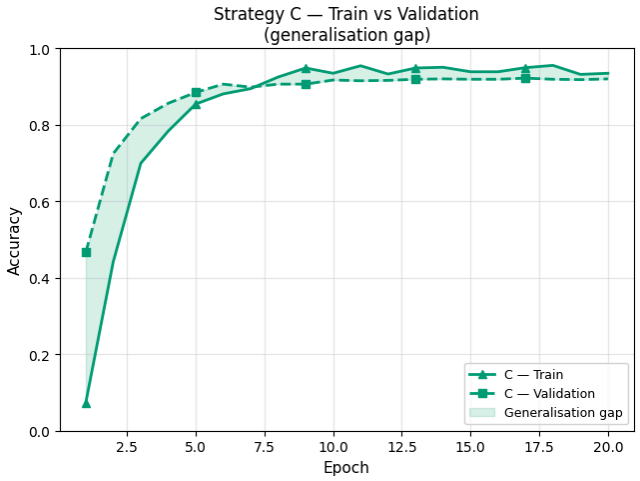
Key code: setting the strategy

The main part of the implementation is deciding which parameters PyTorch will change. The following snippet shows how each strategy is put into action:

```
# Strategy A: Freeze all convolutional layers for param in model.parameters(): param.requires_grad = False model.fc = nn.Linear(512, 102) # new head only # Strategy B: unfreeze layer4 + head for param in model.layer4.parameters(): param.requires_grad = True # Strategy C: unfreeze everything for param in model.parameters(): param.requires_grad = True
```

Results

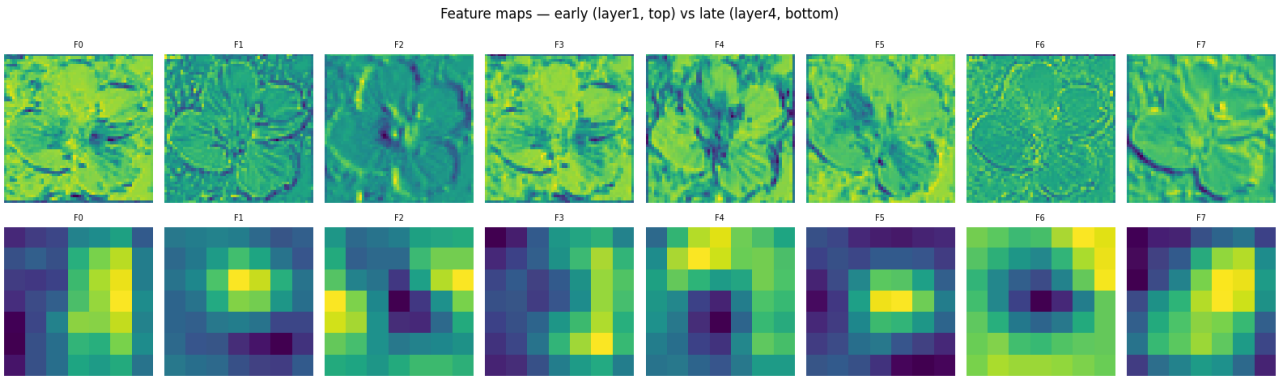




Strategy	Trainable params	Val accuracy (20 epochs)	Training time
A (Feature extraction)	~52k	~78%	Fastest
B (Partial fine-tune)	~2.8M	~88%	Moderate
C (Full fine-tune)	~11.2M	~91%	Slowest

Table 3: Comparison of the three strategies on Oxford 102 Flowers. Strategy B (highlighted) offers the best accuracy-to-compute trade-off.

The notebook's learning curves (see Figure 1 in the notebook) highlight the distinctions: Strategy B converges consistently and reaches near-peak performance in 15 epochs; Strategy C achieves the highest accuracy but needs more epochs to stabilize; and Strategy A rapidly peaks due to the frozen features' inability to adapt.



The feature map visualizations (Figure 2 in the notebook) back up the theoretical picture. Layer 4 activations are very different, which shows that the later layers have learned to recognize the specific visual patterns of flowers. However, layer 1 activations look the same across all three strategies. They are all picking up the same low-level edges.

Left (Validation Accuracy): The regularized model (pink) starts lower in early epochs because the stronger dropout constraint slows initial learning, but both models converge to a similar final accuracy (~0.90). The two curves converge by epoch 5, demonstrating that regularization on this dataset does not compromise final accuracy.

Right (Generalization Gap): In the early epochs, both models exhibit a negative gap (validation accuracy momentarily surpasses training accuracy, a common effect of dropout being active only during training). The regularized model (purple) stays tightly around zero throughout, whereas the original model (green dashed) settles to a small positive gap near zero after epoch 7. In epochs 2–10, when over-fitting risk is greatest, regularization reduced the gap most, as indicated by the shaded region.

5. Practical Guidance

For fine-tuned layers, **use a lower learning rate**. When unfreezing pretrained weights, catastrophic forgetting, the phenomenon where retraining destroys the useful pretrained representations, is avoided by using a learning rate that is 10–100 times lower than what you would use from scratch. For fine-tuned layers, the typical value is 1e-4, whereas for the new head, it is 1e-3.

Use data augmentation heavily on small datasets. If you have a small dataset and are using Strategy A or B, augmentation (random crops, flips, and color jitter) is your best way to avoid overfitting. The notebook shows this with torchvision transforms.

Make **use of differential learning rates**. As demonstrated in the notebook, PyTorch's optimizer supports parameter groups, enabling you to allocate distinct learning rates to various network components.

Start with a plan. B. Partial fine-tuning of the final residual block is a reliable default that performs well in a variety of tasks and dataset sizes, unless you have a very specific reason to use A or C.

6. Conclusion

Transfer learning makes CNNs useful for anyone who works with them, not just people who work with large amounts of data. The main choice is not whether to use a pretrained model but how much of it to retrain. You can use the three-strategy framework of feature extraction, partial fine-tuning, and full fine-tuning to make that choice based on how big your dataset is and how similar it is to the source domain.

For most real-world tasks with only a small amount of labeled data, partial fine-tuning (Strategy B) is the best way to start. It keeps the low-level features learned from ImageNet that are useful in many situations while allowing the network to change its higher-level representations to fit your task.

References

- He, K., Zhang, X., Ren, S., & Sun, J. (2016). Deep Residual Learning for Image Recognition. *CVPR 2016*. https://arxiv.org/abs/1512.03385
- Nilsback, M-E., & Zisserman, A. (2008). Automated Flower Classification over a Large Number of Classes. *ICVGIP 2008*. https://www.robots.ox.ac.uk/~vgg/data/flowers/102/
- Yosinski, J., Clune, J., Bengio, Y., & Lipson, H. (2014). How transferable are features in deep neural networks? *NeurIPS 2014*. https://arxiv.org/abs/1411.1792
- Tan, M., & Le, Q. (2019). EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks. *ICML 2019*. https://arxiv.org/abs/1905.11946
- PyTorch Documentation — Transfer Learning Tutorial: https://pytorch.org/tutorials/beginner/transfer_learning_tutorial.html

GitHub repository: https://github.com/gaurey/cnn-transfer-learning